Revision 06-08-09 · Benchmark Statistics, Streaming & the GPU Dashboard

Interactive revision of lesson 06, lesson 08 and lesson 09: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · Tail latency in a 20-call agent MISSED

Covers: 06

> **QUESTION**
>
> An agent does a task by making 20 LLM calls in a row. The API promises P99 = 2 s. What fraction of tasks have at least one slow call, and how should per-call latency targets be set for agents?

What P99 means. Line up 100 calls fastest to slowest; the 99th is the P99. "P99 = 2 s" means 99 of 100 calls finish under 2 s and 1 is slower, so a single call has a **1% chance** of being slow.

The trap. "1% of calls are slow, so 1% of tasks are slow" is wrong: a task is **20 calls**, and it's slow if **any one** of them is slow. That's 20 chances, not 1.

The trick: flip it to "nothing goes wrong". Counting "at least one slow" directly is messy (it could be the 1st call, the 5th, or two of them). Counting the one clean case, all 20 fast, is easy, so compute that and subtract from 1:

1. One call is fast with probability **0.99**.
2. All 20 fast = 0.99 × 0.99 × … (20 times) = $\mathbf{0.99^{20} \approx 0.82}$.
3. At least one slow = 1 − 0.82 ≈ **18% of tasks**.

So a "1% slow" API makes **18% of agent tasks** hit a slow call, because 20 calls multiply the risk ~20×.

Part 2, per-call targets. To keep slow *tasks* at 1%, each call must be far better than 1%: divide the task target by the number of calls, 1% ÷ 20 ≈ 0.05%, a **P99.95** per-call target. Agents that chain many calls need much tighter per-call promises than single-shot apps, or fewer calls.

Computing 0.99^{20} by hand

- Rough: "lose 1% twenty times" ≈ 20 × 1% = 20% lost, so ≈ 0.80 survives.
- Better: $(1 - x)^n \approx e^{\wedge}(-n{\cdot}x) = e^{\wedge}(-0.2) \approx 1 - 0.2 + 0.02 =$ **0.82** (using $e^y \approx 1 + y + y^2/2$).
- Exact: square repeatedly: $0.99^2 = 0.9801 \to {}^4 \approx 0.9606 \to {}^8 \approx 0.9227 \to {}^{16} \approx 0.8515$; then ${}^{16} \times {}^4 \approx 0.8515 \times 0.9606 \approx 0.818$.

> **RE-DRILL**
>
> For "at least one" questions, compute the chance of "none" and subtract from 1. Remember $(1 - x)^n \approx e^{\wedge}(-n{\cdot}x)$: 50 calls at P99 → $e^{\wedge}(-0.5) \approx 0.61$ → ~39% of tasks see a slow call.

Q2 · Is a 3.5% speedup real? MISSED

Covers: 06

QUESTION

You tweak a setting and benchmark TPOT (time per output token). Before: 5 runs averaging 20.0 ms. After: 5 runs averaging 19.3 ms. Each run wobbles by about σ ≈ 0.8 ms. A teammate says "3.5% faster, ship it." Is it real?

The core idea: measurements wobble. Run the exact same benchmark twice and the number jitters. σ = 0.8 ms means a single run typically lands ~0.8 ms from the truth. The gap we saw is only 20.0 − 19.3 = 0.7 ms, smaller than one run's wobble, so maybe it's a real speedup, maybe it's just noise. We need to tell those apart.

Why averaging helps, and why only by √n. Each run misses the truth by a random amount, sometimes high, sometimes low, so averaging makes the misses partly cancel. Not perfectly (that would make the average exact) and not never (that would make averaging useless): they cancel **randomly**. Adding n random ±wobbles lands you about √n × σ away (the random-walk fact), and the average is that sum ÷ n, so the average's wobble is **σ ÷ √n**. For 5 runs: 0.8 ÷ √5 ≈ 0.8 ÷ 2.2 ≈ **0.36 ms** (the "standard error"). Halving it costs 4× the runs, not 2×.

Now build the noise band step by step:

1. **One run** wobbles by σ ≈ **0.8 ms**.
2. **An average of 5** wobbles by σ ÷ √5 ≈ **0.36 ms**.
3. **The difference** of two averages wobbles more, because both ends are jittery; two independent wobbles combine by √2 ≈ 1.4×: 0.36 × 1.4 ≈ **0.5 ms**.
4. **"Pretty sure" ≈ 2 wobbles** (~95% confidence): noise band ≈ 2 × 0.5 ≈ **±1.0 ms**.
5. **Compare:** the observed gap is **0.7 ms**, inside ±1.0 ms. **Could be luck: not proven, don't ship yet.**
6. **To settle it:** 4× the runs (20 each) halves the band to ±0.5 ms; if the 0.7 ms gap survives, it's real.

RE-DRILL

Re-derive it as a story: one run → average of n → difference of two → ×2 for confidence.

Q3 · A benchmark claiming 2,000 tokens/s PARTIAL

Covers: 08, 01

QUESTION

Someone times `model.generate()` for Llama 3 8B, batch 1 (one user), one H100, and reports 2,000 tokens/s. Why must it be wrong, and what are two likely bugs?

Why it must be wrong. From lesson 01, batch-1 decode streams all 16 GB of weights per token, and 16 GB ÷ 3.35 TB/s ≈ 4.8 ms/token ≈ **200 tokens/s**. 2,000 is 10× over a hard physical limit (set by **memory bandwidth**, not the math units, which sit idle), so it's a measurement bug, not a breakthrough.

Bug 1 · no GPU synchronization. The GPU runs **asynchronously**: calling `generate()` just **queues** the work and Python returns immediately. If you stop the timer right there, you measured the *hand-off*, not the GPU finishing, so the time is tiny and tokens/s is huge. Fix: `torch.cuda.synchronize()` (or CUDA events) before reading the clock, so you wait for the GPU.

Bug 2 · counting the prompt as output. Dividing the *whole* sequence (prompt + generated) by the time instead of only the generated tokens inflates the rate (1.5× too high in lesson 08's example).

Fix: synchronize, count only generated tokens, and report TTFT and TPOT separately.

> **RE-DRILL**
>
> Sanity-check every benchmark against the floor before believing it.

Q4 · "GPU-util is 100%, we need more GPUs" PARTIAL

Covers: 09

> **QUESTION**
>
> One user chats with Llama 3 8B on an H100. Dashboard: GPU-util 100%, power 350 W of a 700 W cap, decode 6 ms per token. The manager wants more GPUs. Is it maxed out? Is half the power cap a problem? How do you measure real usage?

Part 1 · what GPU-util measures. It's a **duty cycle**: the fraction of time that *at least one* kernel was running. An H100 has 132 SMs (compute units); one kernel keeping just one of them busy the whole time still reads **100%**. So 100% means "something was always running", not "maxed out". The manager is misreading it.

Part 2 · is half the power cap a problem? No, it's **normal** for decode. Decode is memory-bound, so the math units spend most of their time **waiting for weight bytes**, and idle math units draw less power. A real power issue shows up as **throttling** in the clock event reasons (power-cap or thermal), not as a low draw.

Part 3 · measure the real usage with MBU. Decode is limited by bandwidth, so measure how much bandwidth you use: **achieved bytes/s ÷ peak bytes/s**.

- Achieved: 16 GB ÷ 6 ms ≈ **2.67 TB/s**.
- MBU = 2.67 ÷ 3.35 ≈ **80%**. (Compute: MFU = (15 GFLOP ÷ 6 ms) ÷ 989 TFLOPS ≈ **0.25%**, almost idle.)

Verdict: not maxed out. One user already uses ~80% of bandwidth with compute idle, so **batching more users is nearly free**. Don't buy GPUs; raise the batch size.

RE-DRILL

MBU = bytes per step ÷ step time ÷ peak bandwidth; MFU = FLOPs per step ÷ step time ÷ peak FLOPS.

Exercise · See benchmark noise by simulation SOLID

Covers: 06

QUESTION

Simulate long-tailed latencies. (A) Estimate P99 from n = 100, 1,000, 10,000 samples, 200 times each, and measure how much the estimates wobble. (B) Brute-force the 20-call agent: what fraction of tasks hit a slow call?

```
def p99_spread(n, repeats=200):
    estimates = [p99([latency() for _ in range(n)]) for _ in range(repeats)]   # 200 benchmarks
    mean = statistics.mean(estimates)
    rel_spread = statistics.stdev(estimates) / mean
    return mean, rel_spread

def slow_task_fraction(calls=20, q=0.01, tasks=100_000):
    slow_tasks = 0
    for _ in range(tasks):
        if any(random.random() < q for _ in range(calls)):   # slow if ANY call is slow
            slow_tasks += 1
    return slow_tasks / tasks
```

Samples per benchmark	P99 estimate	Wobble
100	2.97 s	±16%
1,000	3.17 s	±6%
10,000	3.20 s	±2%

Slow tasks: **18.3%**, matching $1 - 0.99^{20} \approx 18\%$.

What it shows: - Wobble shrinks like √n: 100× the samples → ~8× less wobble (the σ/√n from Q2). - Small samples **under-report the tail**: at n = 100 only ~1 sample lands beyond P99, so the estimate averages 2.97 s vs the true ~3.20 s. A 100-request benchmark flatters your tail.